

week-05-javascript-fundamentals

Week 05 — JavaScript Fundamentals

Session goal: by the end of class you have a working tip calculator in Node — variables, functions, arrays and objects — written by you, not copied from slides.

JavaScript adds behaviour to your programs. Today you write pure `.js` files and run them with `node`. No browser.

Warm-up (5 min)

In a new file `warmup.js`, without looking anything up, write and run:

1. A `const` for your name and a `let` for a score starting at `0`
2. A function `double(n)` that returns `n * 2`
3. An array of three skill strings; `console.log` the second one

```
> Loading Bash code...
```

If you get stuck, that is the gap this week fills.

Variables and Types

Store values with `const` and `let`. Check types with `typeof`.

Example

```
> Loading JavaScript code...
```

Example Explained

Rule: `const` by default. `let` only when you reassign. Never `var`.

Only these are **falsy**: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`.

`"0"`, `"false"`, `[]` and `{}` are **truthy**.

```
> Loading JavaScript code...
```

Note: `const` blocks **reassignment**, not **mutation**. `const user = { name: "A" }` — you cannot do `user = {}`, but you can do `user.name = "B"`.

Class activity A — `const` / `let` / `===` drill (8 min)

1. Create `week-05/basics.js`
2. Declare `const course = "Web Eng"` and try `course = "Other"` — confirm it throws
3. Use `let count = 0`, then increment it twice; `console.log(count)`
4. Predict, then run: `console.log(0 == false)`, `console.log(0 === false)`, `console.log([] == false)`
5. Write a one-line comment under each result explaining why

Done when: `node basics.js` prints your predictions vs reality, and you never used `var`.

Strings, Numbers, and Functions

Build small tools with template literals and arrow functions.

Example — Strings and Numbers

```
> Loading JavaScript code...
```

Example — Functions

```
> Loading JavaScript code...
```

Example Explained

- Prefer template literals over `+` for strings
- `toFixed()` returns a **string** — keep numbers as numbers until display
- Arrow functions: implicit return for one expression; explicit `return` inside `{ }`
- Returning an object from an arrow needs parentheses: `() => ({ name })`

Warning: `toFixed()` returns a string. `"10.00" + 5` is `"10.005"`.

Class activity B — three functions (12 min)

In `basics.js` (or a new `helpers.js`), implement and call:

1. `getInitials("Ayesha Khan")` → `"AK"`
2. `isPalindrome("Race car")` → `true` (ignore case and spaces)
3. `countWords("the quick brown fox")` → `4`

Done when: all three `console.log` the expected values with `node`.

Conditionals and Safe Shortcuts

Choose paths with `if` / `else`. Prefer `??` and `?.` for safe defaults.

Example

```
> Loading JavaScript code...
```

Example Explained

Tool	Use
<code>if / else if</code>	Multiple branches
<code>? :</code>	Short two-way choice
<code> </code> <code> ?? </code> Fallback only for null / undefined <code> ?.</code>	Safe property access — no crash if missing

Tip: If `0` is a valid value, use `??`, not `||`.

Class activity C — grade + nullish (8 min)

1. Write `letterGrade(score)` returning `"A" – "F"` with `if / else if`
2. Write `displayCount(n)` that shows `n` unless `n` is `null / undefined` — use `??`, not `||`
3. Test with `0`, `null`, and `42` — only `null` should fall back

Done when: `displayCount(0)` prints `0`, not a default.

Arrays

Arrays hold ordered lists. Prefer non-mutating methods.

Example

```
> Loading JavaScript code...
```

Example Explained

Method	Meaning	Mutates?
<code>map</code>	One output per input	No
<code>filter</code>	Keep items that pass a test	No
<code>find</code>	First match (or <code>undefined</code>)	No
<code>reduce</code>	Fold list into one value	No
<code>push</code> / <code>pop</code> / <code>splice</code> / <code>sort</code>	Change in place	Yes

Prefer non-mutating methods — React will require it later.

Tip: `map` = one output per input. `filter` = narrow the list. Master these two first.

Class activity D — students pipeline (12 min)

Using the `students` array above (copy it into `arrays.js`):

- Names of students above the class average (use `reduce` for the average, then `filter` + `map`)

2. `students.some(s => s.score > 90)` and `students.every(s => s.score > 40)` — log both
3. Sort a **copy** high → low by score — do not mutate the original
(`[...students].sort(...)`)

Done when: average names print correctly, and `console.log(students)` still shows the original order.

Objects

Objects store named data. Destructure and spread to work with them cleanly.

Example

```
> Loading JavaScript code...
```

Example Explained

- Destructuring pulls properties into variables
- Defaults fill missing keys: `{ role = "student" }`
- Spread `{ ...base, role: "admin" }` builds a new object
- Spread is a **shallow** copy — nested objects stay shared (`structuredClone` for deep copy)

Warning: Spread is a **shallow** copy. Nested objects stay shared.

Class activity E — products + groupBy (10 min)

1. `inStock` titles via `filter` + `map`
2. Total price via `reduce`
3. `groupBy(products, "category")` with `reduce` → `{ peripherals: [...], cables: [...] }`

Done when: `groupBy` returns an object of arrays keyed by category.

Tip Calculator

Today's build artifact: a validated bill calculator.

Example

```
> Loading JavaScript code...
```

Example Explained

Part	Meaning
Destructured params	Clear named inputs with defaults
Validation + <code>throw</code>	Bad input fails loudly
<code>try / catch</code>	Print errors without crashing the process
<code>Number(...toFixed(2))</code>	Round money for display, keep as numbers

Class activity F — ship the calculator (15 min)

1. Create `calculator.js` with `calculateBill` as above
2. Log a happy-path call (`2400` , `10%` , `3` people)
3. Wrap two failing calls in `try/catch` : bad amount, `people: 0`
4. Run `node calculator.js` — success object + two clear error messages

Done when: one success log and both errors print without crashing the process.

Lab (in class) — finish the fundamentals file

Polish into one coherent `lab.js` (you may import helpers or keep everything in one file). Checklist:

- ☐ `getInitials`, `isPalindrome`, `countWords`
- ☐ Above-average student names via array methods (no `for` loops for that task)
- ☐ `groupBy(products, "category")` with `reduce`
- ☐ `calculateBill` with validation + a passing test for each error case
- ☐ `const` / `let` / `===` / `??` used correctly — no `var`, no `==` in your logic

Pass criteria: `node lab.js` runs with zero uncaught errors; tasks using lists use `map` / `filter` / `reduce`, not classic `for` loops.

Submit: the `.js` file(s) plus a terminal screenshot of a successful run.

Homework

Due before Week 06. Submit in the class channel / LMS as listed for each task.

Task 1 — Gradebook script

Write `gradebook.js` that, given an array of student objects `{ name, score }`, prints: class average, highest scorer, lowest scorer, pass rate (%), and names sorted high → low. Use only array methods.

Submit: `gradebook.js` and a terminal screenshot of the output.

Task 2 — Predict before you run

Write your expected output for each line in a text note, then run and explain any surprises:

```
> Loading JavaScript code...
```

Submit: predictions + short explanations (where reality differed).

Task 3 — freeCodeCamp practice

Complete freeCodeCamp — JavaScript Algorithms and Data Structures, **Basic JavaScript** section.

Submit: screenshot of completion.

Task 4 — Tip calculator edge cases

Extend `calculator.js` with: default tip 15%, reject non-number `tipPercent`, and a helper `formatBill(result)` that returns a readable multi-line string for the console.